

Adversarial Search

AL-2002 Artificial Intelligence Spring 2026

Instructor: Ms. Zill-E-Huma



Adversarial Search



Overview

Adversarial search is a technique in AI that deals with decision-making problems where multiple agents compete against each other.

In **adversarial search**, the AI agent evaluates all possible moves and counter-moves by the opponent to identify the best strategy. This method ensures that the AI can anticipate the opponent's actions and adjust its approach accordingly.

Significance in AI

Adversarial search is widely used in:

- **Game-playing AI:** For example, chess engines that predict the opponent's next moves.
- **Strategy-based systems:** Applications like financial modeling or military simulations.
- **Negotiation AI:** Where two parties aim to maximize their respective outcomes.



Key Contributions of Adversarial Search:

1. Strategic Decision-Making
2. Handling Uncertainty:
3. Application in Competitive Systems:
 - Business negotiations: Predicting competitor strategies.
 - Autonomous vehicles: Handling interactions with other agents on the road.
 - AI-driven financial models: Anticipating market changes.



Adversarial Search Algorithm

- MiniMax
- Alpha-Beta Pruning

MiniMax Algorithm

The Minimax Algorithm is a foundational adversarial search algorithm used in turn-based games. It aims to **minimize** the possible **loss for a worst-case** scenario while **maximizing potential gains**. In other words:

- **The Maximizer:** Tries to maximize the score.
- **The Minimizer:** Tries to minimize the score.

Key Terminologies in the Minimax Algorithm:

1. **Minimax Tree:**

- A decision tree representing all possible moves of both players.
- Each level alternates between the maximizer and minimizer.

2. **MAX (Maximizer):**

- The player aiming to maximize the score.
- Focuses on selecting the move with the highest value.

3. **MIN (Minimizer):**

- The player aiming to minimize the score.
- Focuses on reducing the opponent's score.

4. **Initial State:**

The starting point of the game or scenario.

5. **Terminal State:**

The end of the game, where the outcome (win, lose, or draw) is evaluated.

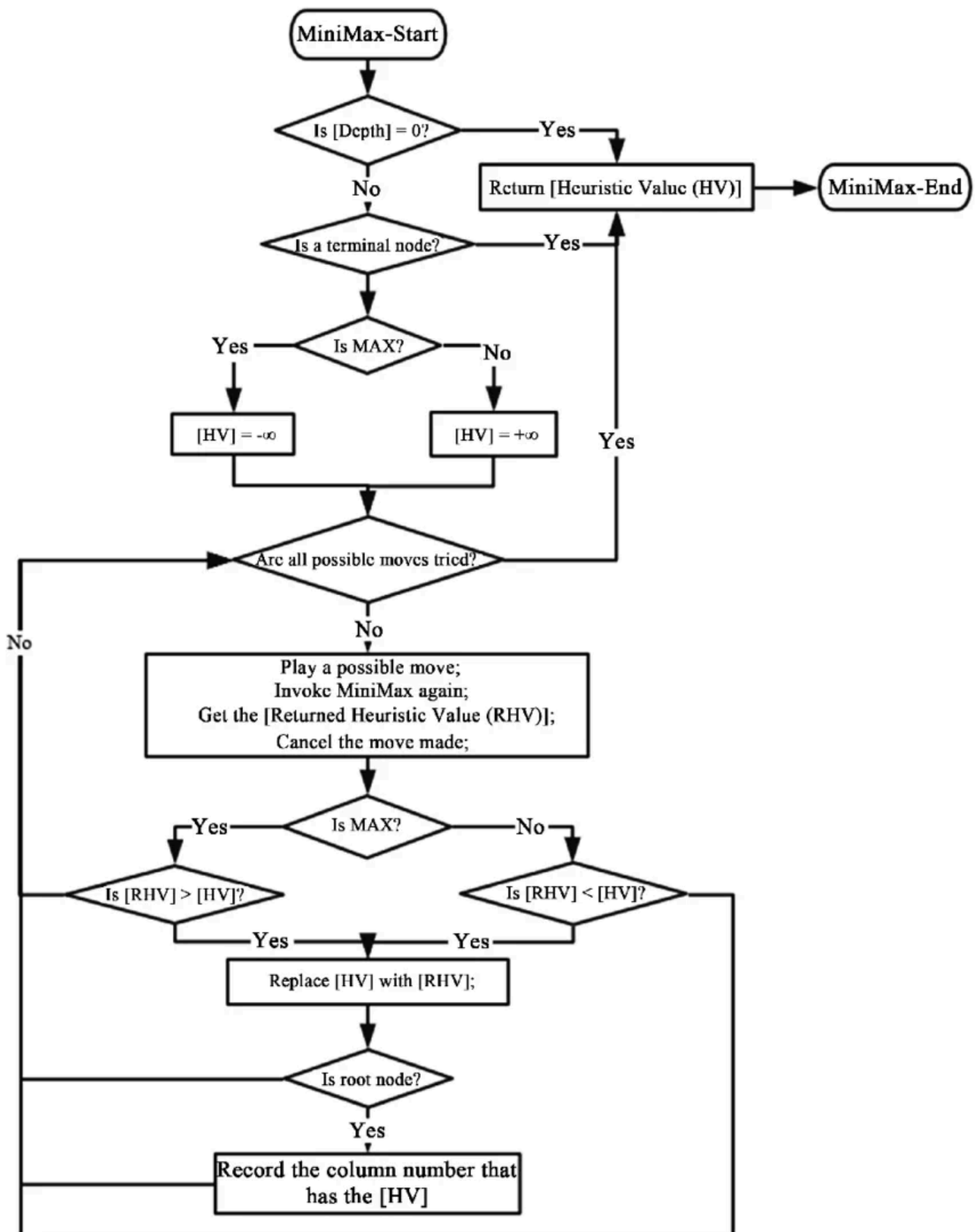
6. **Heuristic Evaluation Function:**

A function used to estimate the value of non-terminal states when the search cannot explore the entire tree due to computational limitations.

Example

Tic-Tac-Toe

-
- Level 0**
X Plays Next
X maximizes
- Level 1**
O Plays Next
O minimizes
- Level 2**
X plays Next
X maximizes
- Level 3**
Terminal
- The diagram shows a search tree starting from a root node (Level 0) representing X's move. It branches into three nodes for O's response (Level 1). Each of these branches into two nodes for X's next move (Level 2). Finally, there are six terminal nodes (Level 3) showing the board state after X's move. Red arrows indicate the path chosen by the algorithm, which selects the maximum value at each level where it is X's turn to move.
- | | | |
|---|---|---|
| O | X | O |
| | X | |
| X | O | |
- oMovesCount = 3
-
- | | | |
|---|---|---|
| O | X | O |
| X | X | |
| X | O | |
- oMovesCount = 3
- | | | |
|---|---|---|
| O | X | O |
| | X | |
| X | O | |
- oMovesCount = 3
- | | | |
|---|---|---|
| O | X | O |
| | X | |
| X | O | |
- oMovesCount = 3
- | | | |
|---|---|---|
| O | X | O |
| X | X | O |
| X | O | |
- oMovesCount = 4
- | | | |
|---|---|---|
| O | X | O |
| X | X | |
| X | O | O |
- oMovesCount = 4
- | | | |
|---|---|---|
| O | X | O |
| O | X | X |
| X | O | |
- oMovesCount = 4
- | | | |
|---|---|---|
| O | X | O |
| | X | X |
| X | O | O |
- oMovesCount = 4
- | | | |
|---|---|---|
| O | X | O |
| O | X | |
| X | O | X |
- oMovesCount = 4
- | | | |
|---|---|---|
| O | X | O |
| | X | O |
| X | O | X |
- oMovesCount = 4
- | | | |
|---|---|---|
| O | X | O |
| X | X | O |
| X | O | O |
- oMovesCount = 4
- | | | |
|---|---|---|
| O | X | O |
| X | X | X |
| X | O | O |
- oMovesCount = 4
- | | | |
|---|---|---|
| O | X | O |
| O | X | X |
| X | O | X |
- oMovesCount = 4
- | | | |
|---|---|---|
| O | X | O |
| X | X | X |
| X | O | O |
- oMovesCount = 4
- | | | |
|---|---|---|
| O | X | O |
| O | X | X |
| X | O | X |
- oMovesCount = 4
- | | | |
|---|---|---|
| O | X | O |
| X | X | O |
| X | O | X |
- oMovesCount = 4
- Draw X wins Draw X wins Draw Draw



Algorithm

- **Initial state:** It specifies how the game is set up at the start (**initial board position + information about whose move it is**).
- **Player(s):** It specifies which player has moved in the state space.

- **Action(s):** It returns the set of legal moves in state space (**legal moves a player can make**).
- **Result(s, a):** It is the transition model, which specifies the result of moves in the state space.
- **Terminal-Test(s):** Terminal test is true if the game is over, else it is false in any case. The state where the game ends is called terminal states.
- **Utility(s, p):** A utility function gives the final numeric value for a game that ends in terminal states s for player p (**measures the outcome of the game and its desirability**). It is also called the payoff function. For Chess, the outcomes are a win, loss, or draw and its payoff values are +1, 0, ½. And for tic-tac-toe, utility values are +1, -1, and 0.

Minimax Pseudocode

```
function MINIMAX(node, depth, maximizingPlayer):
    if depth == 0 or node is a terminal state:
        return heuristic value of node

    if maximizingPlayer:
        maxEval = -∞
        for each child of node:
            eval = MINIMAX(child, depth - 1, False)
            maxEval = max(maxEval, eval)
        return maxEval
    else:
        minEval = +∞
        for each child of node:
            eval = MINIMAX(child, depth - 1, True)
            minEval = min(minEval, eval)
        return minEval
```

Finding the Best Move

```
function findBestMove(board):
    bestMove = None
    bestValue = -∞

    for each move in board:
        value = MINIMAX(move, depth, False)
        if value > bestValue:
            bestValue = value
            bestMove = move
```

```
return bestMove
```

```
import math

# Initialize board
board = [
    [' ', ' ', ' '],
    [' ', ' ', ' '],
    [' ', ' ', ' ']
]

# Check winner
def check_winner(b):
    # Rows
    for row in b:
        if row[0] == row[1] == row[2] and row[0] != ' ':
            return row[0]

    # Columns
    for col in range(3):
        if b[0][col] == b[1][col] == b[2][col] and b[0][col] != ' ':
            return b[0][col]

    # Diagonals
    if b[0][0] == b[1][1] == b[2][2] and b[0][0] != ' ':
        return b[0][0]

    if b[0][2] == b[1][1] == b[2][0] and b[0][2] != ' ':
        return b[0][2]

    return None

# Check if moves left
def is_moves_left(b):
    for row in b:
        if ' ' in row:
            return True
    return False

# Minimax function
def minimax(b, depth, is_max):
    winner = check_winner(b)

    if winner == 'X': # AI
        return 1
    elif winner == 'O': # Human
        return -1
    elif not is_moves_left(b):
        return 0

    if is_max:
```

```

        best = -math.inf

        for i in range(3):
            for j in range(3):
                if b[i][j] == ' ':
                    b[i][j] = 'X'
                    best = max(best, minimax(b, depth + 1, False))
                    b[i][j] = ' '

            return best

    else:
        best = math.inf

        for i in range(3):
            for j in range(3):
                if b[i][j] == ' ':
                    b[i][j] = 'O'
                    best = min(best, minimax(b, depth + 1, True))
                    b[i][j] = ' '

            return best

# Find best move for AI
def find_best_move(b):
    best_val = -math.inf
    best_move = (-1, -1)

    for i in range(3):
        for j in range(3):
            if b[i][j] == ' ':
                b[i][j] = 'X'
                move_val = minimax(b, 0, False)
                b[i][j] = ' '

                if move_val > best_val:
                    best_move = (i, j)
                    best_val = move_val

    return best_move

# Print board
def print_board(b):
    for row in b:
        print("|".join(row))
        print("-" * 5)

# Game loop
while True:
    print_board(board)

    # Human move
    x = int(input("Enter row (0-2): "))

```

```

y = int(input("Enter col (0-2): "))

if board[x][y] != ' ':
    print("Invalid move!")
    continue

board[x][y] = 'O'

if check_winner(board) == 'O':
    print_board(board)
    print("You win!")
    break

if not is_moves_left(board):
    print_board(board)
    print("Draw!")
    break

# AI move
move = find_best_move(board)
board[move[0]][move[1]] = 'X'

if check_winner(board) == 'X':
    print_board(board)
    print("AI wins!")
    break

if not is_moves_left(board):
    print_board(board)
    print("Draw!")
    break

```

```

| |
-----
| |
-----
| |
-----
Enter row (0-2): 0
Enter col (0-2): 2
| |O
-----
|X|
-----
| |
-----
Enter row (0-2): 0
Enter col (0-2): 1
X|O|O
-----
|X|
-----
| |

```